

Problem A. Coprime Array

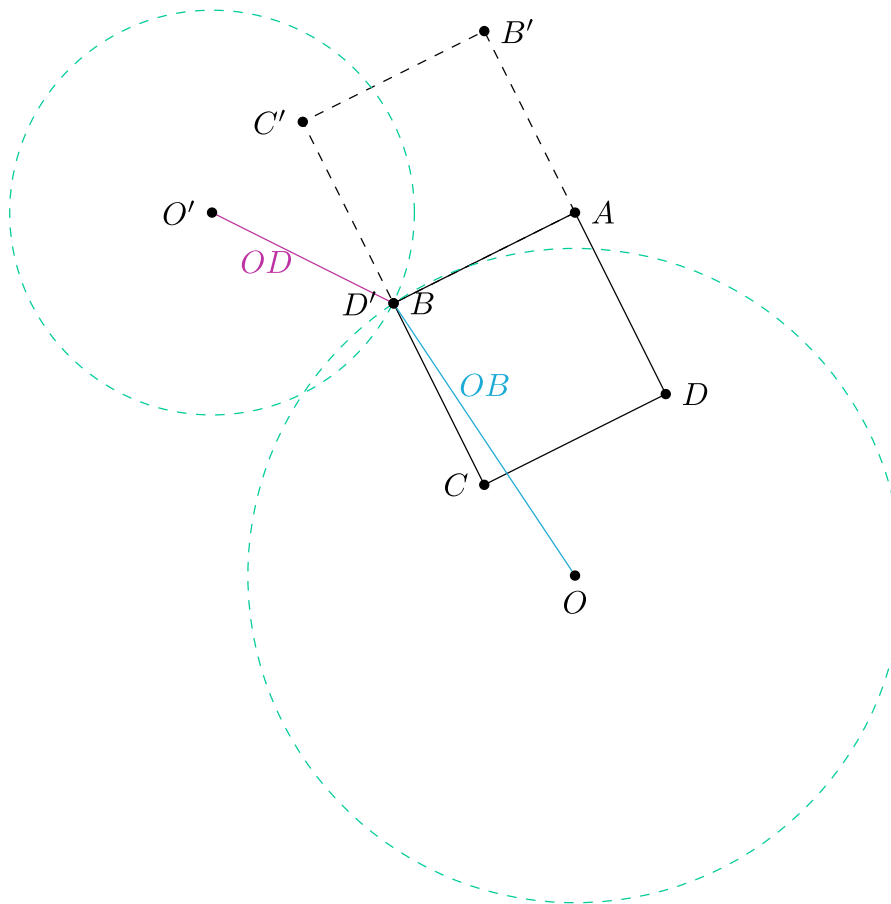
Let's show that the length of the array is at most 3 (and it is always possible). It's easy to check if the answer is 1, so we assume that this is not the case.

If x is even, then all elements of the array are odd, so if s is odd then we will add 1 to the array and solve the problem for $s - 1$ and x (and it's clear two elements are not enough for the same reason).

If x is odd or s is even, we claim that a length 2 array is enough. To prove this, we will provide the construction: for each prime factor p in the prime factorization of x , there exists an integer d such that $d \not\equiv 0, m \pmod{p}$, by the hypothesis. Then combining all these congruences over all prime factors p , which exists by using the Chinese Remainder Theorem, yields the desired construction.

Problem B. Square Locator

If we rotate the square and the origin 90° about A , then we can see that the origin goes to the point O' such that $BO' = DO$. Thus we can draw the circles with radii BO centered at O and DO centered at O' , at least one of the locations they intersect at will be B . Since two circles have not more than two intersections, there are now only finitely many cases to check, and we can check them all.



Problem C. -is-this-bitset-

This problem is basically a knapsack problem with a twist. You can freely change some of the knapsack weights yourself. Without changing any weights, the problem could be solved by running a knapsack DP with bitsets, while doing DFS on the tree, but this takes a lot of memory and is pretty slow. Maybe with lots of constant optimization and some memory complexity optimizations, this could be fitted into the time limit and memory limit, but this should be hard.

The main idea is that because the tree in the input is binary, there are not too many nodes at depth ≤ 11 . There are $2^{12} - 1$ of such nodes, and we can change all the weights to $2^{\text{my depth}}$ on each of these nodes.

This makes sure that after we leave the low depths, our knapsack will contain a big interval of values that are all possible as a subset sum of length $\approx 2^{12}$. Notice that this power of 2 can always be chosen to be $\Theta(k)$, where k is the number of weights we are allowed to change.

With bitsets, this does not really help. But storing the simple dynamic programming in a run-encoded form, where only the intervals of 1 are saved, the representation of the DP state is of size $O(A/k)$, because each interval is of size at least 2^{12} . Transitions using these DP states can be done with some merges in the same time complexity, which gives a fast enough solution in time $O(nA/k)$.

Unfortunately, this uses too much memory naively, as sometimes we need to store a DP representation for each node along a long path, giving also $O(nA/k)$ memory.

So we design a datastructure where we can add a weight to the knapsack, and rollback to the previous state, with reduced memory usage, while still keeping the same time complexity. We can actually make the memory usage of this datastructure $O(n + A)$ which fits easily into the memory limit, and then use this datastructure while walking along the tree.

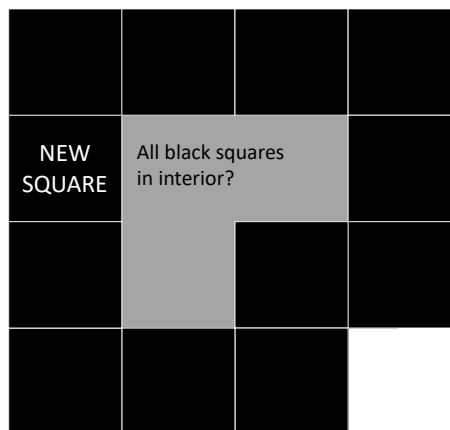
The datastructure consists of the current representation which is again the run-encoded form of the current DP. And a stack, which stores for each node on the root-to-node path, all the DP positions which were changed from a 0 to a 1 at node u . This stack has size at most A at any point in the algorithm, but pushing and popping items to this stack could need $O(A)$ time for each node in the tree, blowing up the complexity. But each time a weight a_u is included in the knapsack, there are only $O(A/k)$ intervals which are newly added. So we only need to push that many intervals to the stack.

So in the end, we have a solution in time $O(nA/k)$ and memory $O(n + A)$.

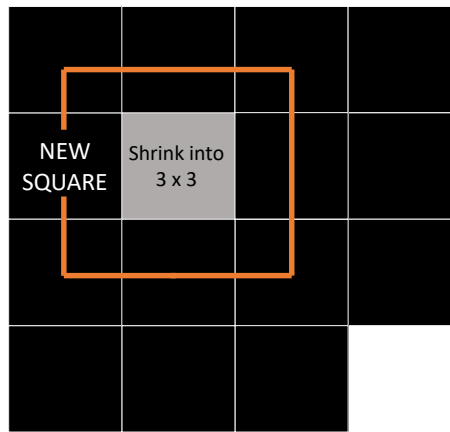
Problem D. Cycle Game

This problem was inspired by the game Havannah, which is a boardgame on a hexagonal grid. To make it more CP-friendly, I chose for a normal grid this time. The ending condition of a cycle with non-empty interior is one of the winning positions in Havannah. So actually in this problem, you are implementing a game engine for a simplified variant of Havannah, which checks if the moves that are played are winning or not.

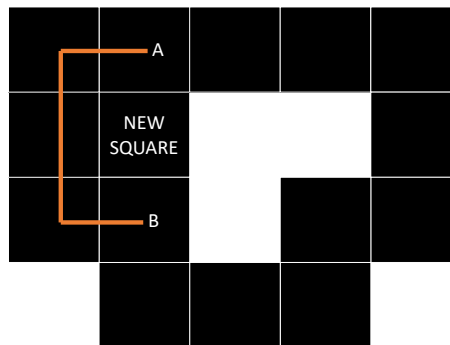
To solve it, we can do casework on what happens when we make one more new square black. Some initial observations: If it connects to not more than one black square, it cannot make any new cycles. More generally, if we only connect to at most 1 square of each previous connected component of black squares, we also do not create any new cycles. All components until now can only have cycles which have no interior, which means that their “width” is at most 2: you can’t have any 3×3 square completely filled, as this would make for a game-ending cycle of size 8. We need to check two cases: Either the game-ending cycle in its interior only contains black squares, or it contains at least one white square.



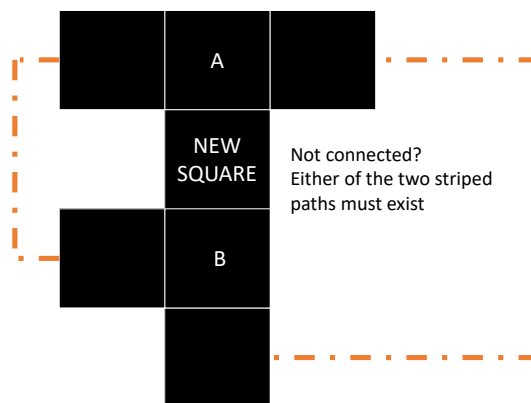
Two nodes A and B adjacent to the new square, and already in the same component.



Interior of the cycle is fully black, reduce to 3×3



Interior of the cycle has a white square, case 1



Interior of the cycle has a white square, case 2

If the game-ending cycle contains a white square on the inside, a white square has to be 8-adjacent (so diagonals count too) to the new black square, as otherwise, there was already an earlier cycle which ended the game. We can loop over all the empty squares that are 8-neighbours of the new square. We can also loop over the two neighbours of the new black square that will complete the cycle. They must be already in the same component, and looking at only the 8 8-adjacent neighbours of the new square, they should not be already connected. If this were the case, then the cycle where you replace the new black square, with the other path through the neighbours of the new square, would also enclose the white square so this would already have created a cycle earlier. If the two 4-neighbours of the new square were in the same component, but there's no path in the 8-neighbours, there must exist some path that connects the two black squares, and it must contain one of the white squares that is an 8-adjacency neighbour. So all these cases can be checked by just inspecting 8 neighbours.

If the game-ending cycle contains no white square, then a bigger cycle can always be compressed to a 3×3 cycle. There are only 8 candidate cycles that are 3×3 and include the new black square, so they can be checked with brute force.

The total time complexity is $O((nm+k)\alpha(nm))$, because for gathering connected component information, we can use a DSU.

Problem E. Sum of Squares

The solution that we have in mind works in $O(n^3 + n^2 \log m)$.

For simpler notation, let's decrease m by 1 and work with $D(x_0, \dots, x_m)$ instead of $D(x_1, \dots, x_m)$.

To solve this problem, we should first rewrite $D(\mathbf{x})$.

As a first step, we rewrite the second product, as it is the determinant of the Vandermonde matrix:

$$D(\mathbf{x}) = A(x_0) \dots A(x_m) \begin{vmatrix} 1 & x_0 & \dots & x_0^m \\ 1 & x_1 & \dots & x_1^m \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_m & \dots & x_m^m \end{vmatrix}$$

After doing so, we can as well bring $A(x_0), \dots, A(x_m)$ under the determinant:

$$D(\mathbf{x}) = \begin{vmatrix} A(x_0) & x_0 A(x_0) & \dots & x_0^m A(x_0) \\ A(x_1) & x_1 A(x_1) & \dots & x_1^m A(x_1) \\ \vdots & \vdots & \ddots & \vdots \\ A(x_m) & x_m A(x_m) & \dots & x_m^m A(x_m) \end{vmatrix}$$

In other words, let $A_i(x) = x^i A(x)$, then we can say that the j -th element in the i -th row under the determinant is $A_j(x_i)$. Using the Leibniz formula for determinants, we get

$$D(\mathbf{x}) = \sum_{\sigma \in S_{m+1}} \text{sgn}(\sigma) \prod_{i=0}^m A_{\sigma_i}(x_i).$$

If we denote $[x^j]A_i(x) = a_{ij}$, and expand each A_{σ_i} in the product, this further rewrites as

$$D(\mathbf{x}) = \sum_{\sigma \in S_{m+1}} \text{sgn}(\sigma) \sum_{\mathbf{j}} \mathbf{x}^{\mathbf{j}} \prod_{i=0}^m a_{\sigma_i j_i},$$

which also allows to swap the order of summation and find out that the coefficient near $\mathbf{x}^{\mathbf{j}}$ is

$$[\mathbf{x}^{\mathbf{j}}]D(\mathbf{x}) = \sum_{\sigma \in S_{m+1}} \text{sgn}(\sigma) \prod_{i=0}^m a_{\sigma_i j_i}.$$

Hmm, but it's actually still a determinant, just of a bit different matrix:

$$[\mathbf{x}^{\mathbf{j}}]D(\mathbf{x}) = \begin{vmatrix} a_{0j_0} & a_{0j_1} & \dots & a_{0j_m} \\ a_{1j_0} & a_{1j_1} & \dots & a_{1j_m} \\ \vdots & \vdots & \ddots & \vdots \\ a_{mj_0} & a_{mj_1} & \dots & a_{mj_m} \end{vmatrix}$$

So, our whole problem is to find the sum of squared determinants:

$$s = \sum_{\mathbf{j}} \left| \begin{vmatrix} a_{0j_0} & a_{0j_1} & \dots & a_{0j_m} \\ a_{1j_0} & a_{1j_1} & \dots & a_{1j_m} \\ \vdots & \vdots & \ddots & \vdots \\ a_{mj_0} & a_{mj_1} & \dots & a_{mj_m} \end{vmatrix} \right|^2$$

Let's expand one of the inner determinants via Leibniz rule:

$$s = \sum_{\mathbf{j}} \sum_{\sigma \in S_{m+1}} \text{sgn}(\sigma) a_{\sigma_0 j_0} \cdots a_{\sigma_m j_m} \begin{vmatrix} a_{0j_0} & a_{0j_1} & \cdots & a_{0j_m} \\ a_{1j_0} & a_{1j_1} & \cdots & a_{1j_m} \\ \vdots & \vdots & \ddots & \vdots \\ a_{mj_0} & a_{mj_1} & \cdots & a_{mj_m} \end{vmatrix}$$

We can multiply $\text{sgn}(\sigma)$ with the remaining determinant to reorder its columns, so they adhere to σ :

$$s = \sum_{\sigma \in S_{m+1}} \sum_{j_0} a_{\sigma_0 j_0} \cdots \sum_{j_m} a_{\sigma_m j_m} \begin{vmatrix} a_{\sigma_0 j_0} & a_{\sigma_0 j_1} & \cdots & a_{\sigma_0 j_m} \\ a_{\sigma_1 j_0} & a_{\sigma_1 j_1} & \cdots & a_{\sigma_1 j_m} \\ \vdots & \vdots & \ddots & \vdots \\ a_{\sigma_m j_0} & a_{\sigma_m j_1} & \cdots & a_{\sigma_m j_m} \end{vmatrix}$$

Note that we also changed the order of summation again, so we can now use the multi-linearity of determinant to multiply the k -th column with $a_{\sigma_k j_k}$, which now converts the element in the i -th row and the k -th column into

$$\sum_{j_k} a_{\sigma_i j_k} a_{\sigma_k j_k} = \mathbf{a}_{\sigma_i} \cdot \mathbf{a}_{\sigma_k},$$

the dot product of the vectors $\mathbf{a}_{\sigma_i}$ and $\mathbf{a}_{\sigma_k}$, consisting of the coefficients of $A_{\sigma_i}(x)$ and $A_{\sigma_k}(x)$ correspondingly. So, the full sum rewrites into

$$s = \sum_{\sigma \in S_{m+1}} \begin{vmatrix} \mathbf{a}_{\sigma_0} \cdot \mathbf{a}_{\sigma_0} & \mathbf{a}_{\sigma_0} \cdot \mathbf{a}_{\sigma_1} & \cdots & \mathbf{a}_{\sigma_0} \cdot \mathbf{a}_{\sigma_m} \\ \mathbf{a}_{\sigma_1} \cdot \mathbf{a}_{\sigma_0} & \mathbf{a}_{\sigma_1} \cdot \mathbf{a}_{\sigma_1} & \cdots & \mathbf{a}_{\sigma_1} \cdot \mathbf{a}_{\sigma_m} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{a}_{\sigma_m} \cdot \mathbf{a}_{\sigma_0} & \mathbf{a}_{\sigma_m} \cdot \mathbf{a}_{\sigma_1} & \cdots & \mathbf{a}_{\sigma_m} \cdot \mathbf{a}_{\sigma_m} \end{vmatrix}$$

Under the summation, we have a determinant of a Gram matrix. The determinant doesn't depend on a permutation of vectors on which it was constructed, so the whole sum finally turns into

$$s = (m+1)! \begin{vmatrix} \mathbf{a}_0 \cdot \mathbf{a}_0 & \mathbf{a}_0 \cdot \mathbf{a}_1 & \cdots & \mathbf{a}_0 \cdot \mathbf{a}_m \\ \mathbf{a}_1 \cdot \mathbf{a}_0 & \mathbf{a}_1 \cdot \mathbf{a}_1 & \cdots & \mathbf{a}_1 \cdot \mathbf{a}_m \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{a}_m \cdot \mathbf{a}_0 & \mathbf{a}_m \cdot \mathbf{a}_1 & \cdots & \mathbf{a}_m \cdot \mathbf{a}_m \end{vmatrix}$$

In other words, s is $(m+1)!$ times the determinant of a matrix T , such that T_{ij} is the dot product of the coefficients of $A_i(x) = x^i A(x)$ and $A_j(x) = x^j A(x)$. This result has a very nice interpretation from tensor point of view, which formulates as

$$|\Gamma(v_1, \dots, v_n)| = \|v_1 \wedge \cdots \wedge v_n\|^2,$$

where Γ is the Gram matrix of vectors $v_1, \dots, v_n$, and $v_1 \wedge \cdots \wedge v_n$ is their exterior product.

Returning to the matrix T , any of its elements

$$T_{ij} = [x^{i-j}]A(x)A(x^{-1}) = t_{|i-j|}$$

only depends on $|i-j|$. Thus, T is a symmetric Toeplitz matrix of size $(m+1) \times (m+1)$, moreover, all of its elements are zeros except the ones on the central $2n+1$ diagonals:

$$T = \begin{bmatrix} t_0 & \cdots & t_n & & \\ \vdots & \ddots & & \ddots & \\ t_n & & t_0 & & t_n \\ & \ddots & & \ddots & \vdots \\ & & t_n & \cdots & t_0 \end{bmatrix}$$

We will now learn how to evaluate its characteristic polynomial $T(\lambda) = \det(T - \lambda I)$. If $\mathbf{x}$ is an eigenvector of T for the eigenvalue λ , then we can formulate it as a system of equations

$$\begin{bmatrix} t_0 - \lambda & \cdots & t_n & & \\ \vdots & \ddots & & \ddots & \\ t_n & & t_0 - \lambda & & t_n \\ & \ddots & & \ddots & \vdots \\ & & t_n & \cdots & t_0 - \lambda \end{bmatrix} \begin{bmatrix} x_0 \\ \vdots \\ \vdots \\ \vdots \\ x_m \end{bmatrix} = \mathbf{0}$$

However, we can also embed it into a larger system of equations:

$$\left[\begin{array}{ccc|cccc|} t_n & \cdots & & t_0 - \lambda & \cdots & t_n & & & \\ & \ddots & & \vdots & \ddots & & \ddots & & \\ & & t_n & & t_0 - \lambda & & t_n & & \\ & & & \ddots & & \ddots & \vdots & & \\ & & & & t_n & \cdots & t_0 - \lambda & \cdots & t_n \end{array} \right] \begin{bmatrix} x_{-n} \\ \vdots \\ x_0 \\ \vdots \\ \vdots \\ x_m \\ \vdots \\ x_{m+n} \end{bmatrix} = \mathbf{0}$$

with additional constraint $x_{-n} = \cdots = x_{-1} = x_{m+1} = \cdots = x_{m+n} = 0$. This system of equations defines a linear recurrence of degree $2n$, but rather than being defined by first $2n$ values, it is defined by first n values, which are $x_{-n}, \dots, x_{-1}$, but also by n values $x_{m+1}, \dots, x_{m+n}$.

Let $L : \mathbb{K}[x] \rightarrow \mathbb{K}$ be a linear functional defined as $L(x^k) = x_{k-n}$. Then, we have a system of equations $L(x^0) = \cdots = L(x^{n-1}) = L(x^{m+n+1}) = \cdots = L(x^{m+2n}) = 0$. For this system to make sense, we notice that $L(P(x)) = L(P(x) \bmod C(x))$, where

$$C(x) = t_n + t_{n-1}x + \cdots + (t_0 - \lambda)x^{n+1} + \cdots + t_{n-1}x^{2n} + t_nx^{2n+1}$$

is the characteristic polynomial of the linear recurrence. In this way, we treat $x_{-n}, \dots, x_{-1}$, aka $L(x^0), \dots, L(x^{2n-1})$, as variables, while $L(x^0), \dots, L(x^{n-1}), L(x^{m+n+1}), \dots, L(x^{m+2n})$ as their linear combinations, defined by $L(x^k) = L(x^k \bmod C(x))$.

Thus, $\mathbf{x} = (x_{-n}, \dots, x_{n-1})$ defines an eigenvector of T with an eigenvalue λ if and only if

$$\begin{bmatrix} x^0 \bmod C(x) \\ \vdots \\ x^{n-1} \bmod C(x) \\ x^{m+n+1} \bmod C(x) \\ \vdots \\ x^{m+2n} \bmod C(x) \end{bmatrix} \begin{bmatrix} x_{-n} \\ \vdots \\ x_{-1} \\ x_0 \\ \vdots \\ x_{n-1} \end{bmatrix} = \mathbf{0},$$

where $x^k \bmod C(x)$ means that we're putting the coefficients of this polynomial into the corresponding row. We can now represent this system of equations in a block form

$$\begin{bmatrix} I_n & 0 \\ U_\lambda & V_\lambda \end{bmatrix} \mathbf{x} = \mathbf{0},$$

as $x^0, \dots, x^{n-1}$ evaluate to themselves modulo $C(x)$, meaning that we have the unit matrix I_n in the upper left block. That being said, the determinant of the system is zero if and only if $\det V_\lambda = 0$. In other words, $\det(T - \lambda I) = 0 \iff \det V_\lambda = 0$. In fact, it implies even stronger result

$$\det(T - \lambda I) = (-1)^{mn} t_n^m \det V_\lambda.$$

As the proof for this is a bit technical, we refer you to the paper “Efficient Algorithms for the Evaluation of the Eigenvalues of (Block) Banded Toeplitz Matrices” by D. Bini and V. Pan (proposition 2.1).

Just as a very brief sketch of the proof, the key idea is that both $\det(T - \lambda I)$ and $(-1)^{mn} t_n^m \det V_\lambda$ are polynomials in λ of degree $m + 1$, with leading coefficients both being either 1 or -1 , and with the same set of zeros, establishing the identity between them.

In any case, this result allows us to find $\det T = (-1)^{mn} t_n^m \det V_0$ in $O(n^3)$ after we find V_0 in $O(n^2 \log m)$ via binary exponentiation to find $x^{m+n+1} \bmod C(x)$.

Problem F. Alternating Cycle

Without some observations this problem seems hard to solve even in polynomial time. The big observation that makes everything doable is the following:

If the point set has ≥ 3 points in the interior of the convex hull, then it is always possible to make a length 6 alternating cycle, and this is the smallest possible.

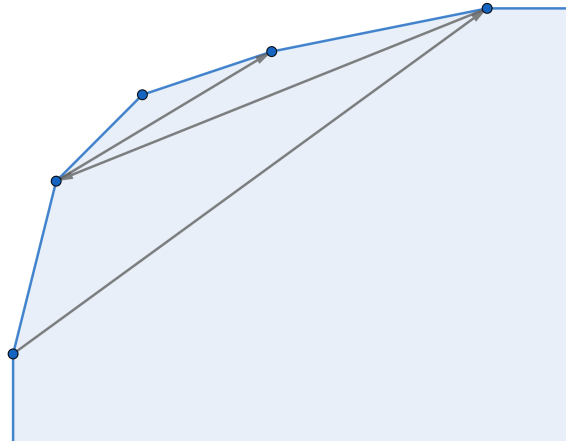
The proof is included at the end of the editorial.

With this observation in hand, the problem becomes tractable: For 0–2 points in the interior, the structure of the point set is pretty simple, and we must exploit this to solve these cases. And for ≥ 3 points in the convex hull, we can remove all but 3 of the interior points. Next, because of the lemma, we can also delete lots of vertices from the convex hull, while these 3 points stay interior. You can prove you can keep doing this, until at most 6 points remain on the hull. On this point set of $O(1)$ points, we can just brute force all 6-tuples to find an alternating cycle.

Some initial observations

It is clear that an alternating cycle has even size, and has size greater than 2. But also for 4 points, it is impossible. If the points are a, b, c, d in order, there are really only two distinct configurations of these points: Either they are in convex position and form a convex polygon of size 4, which can quickly be verified is impossible, or they form a triangle with a point inside, which can also be verified to be impossible. So length 6 is the smallest that could be possible.

Assume there are two points of the convex hull of the point set that are neighbouring in the alternating cycle. If we label the points on the convex hull $h_0, h_1, \dots, h_{k-2}, h_{k-1}$ in cyclic order, we assume the two points neighbouring in the alternating cycle are h_i and h_j , WLOG $i < j$. Assume also that the convex polygon connecting the points $CP = h_i, h_{i+1}, \dots, h_{j-1}, h_j$ does not contain any points in its interior. When the alternating cycle needs to make a counterclockwise or clockwise turn from $\angle h_i h_j$ new, where new must be on the side of CP , then the next point in the alternating cycle, must be another convex hull point, with index $i < o < k$. This actually reduces to exactly the same situation, with a new CP' which has fewer points, and again two points h_j, h_o lying on this convex polygon. This logic repeats, until the convex polygon that we are standing on has only 2 points, and is degenerate, in which case the alternating cycle cannot continue. But this means the alternating cycle cannot have any two neighbours h_i, h_j with CP to one side, that we assumed previously. Let's call this the bad polygon situation.



Bad polygon situation.

0 to 2 interior points

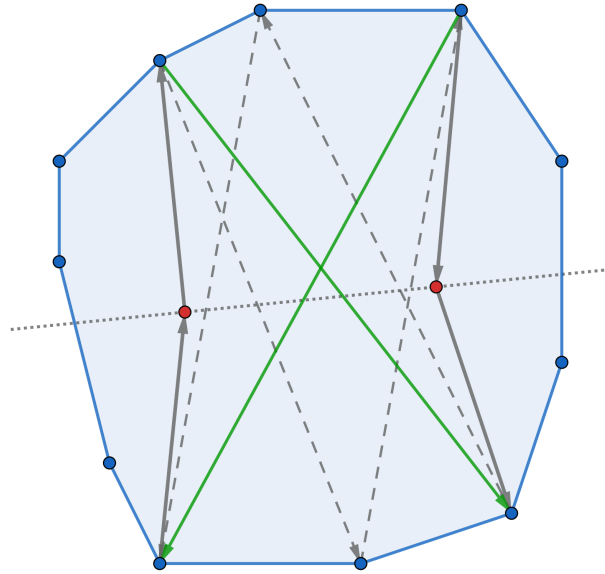
Now let's do casework on the number of points in the interior of the convex hull.

If this number of points is 0, if we look at any two neighbours in the alternating cycle, they immediately form the bad polygon situation. So an alternating cycle is impossible.

If the number of interior points is 1, if we want to make an alternating cycle, it must contain the interior point. But the two next points in the alternating cycle, after this interior point, must be two hull points, and they also form the bad polygon situation, because the one interior point that could be inside the convex polygon, is on the other side of line $h_i \rightarrow h_j$. So an alternating cycle is impossible.

If the number of interior points is 2 it gets more tricky. We can be sure we must use both interior points. We can also see that the two interior points cannot be neighbours in the alternating cycle. This is because again, looking at the two next points after the two interior points, they form the bad polygon situation. To avoid the bad polygon situation altogether, the only way to connect the two interior points is by making the < 180 degree side face outward. Let's call the two interior points i_1 and i_2 . For any two hull points h_a, h_b that are neighbours in the alternating cycle, if both i_1 and i_2 lie to one side of the line $h_a \rightarrow h_b$, then on the other side the bad polygon situation occurs. So i_1 and i_2 must always lie on different sides of such a line, and this means, because h_a and h_b are hull vertices, that the line segment $h_a h_b$ must intersect segment $i_1 i_2$. The general form of a alternating cycle must be as shown in the figure.

But given any such alternating cycle, it can always be shortcut to a cycle of length 6. Divide the points that participate in the alternating cycle into two, the ones that lie above the line $i_1 \rightarrow i_2$ and below the line. Then from both sides, pick the first and the last point. Connect these 4 vertices in a cross and connect to the the two interior points to make a butterfly. These two cross diagonals must also intersect segment $i_1 i_2$, because they are enclosed on both sides, by some other diagonals which also intersected the segment.



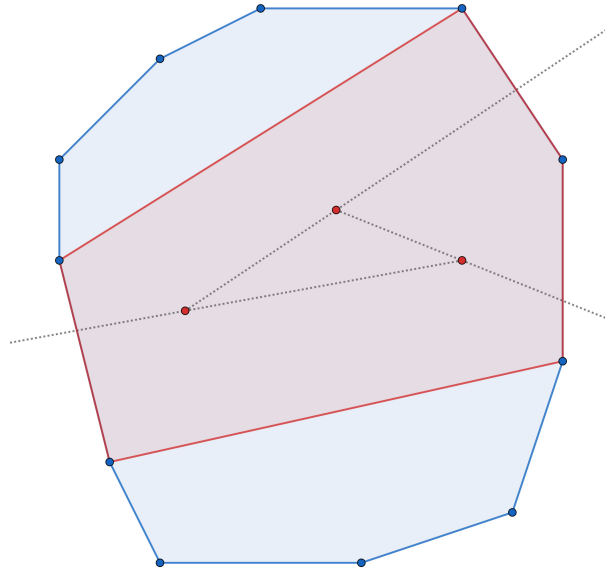
Reducing an arbitrary cycle by taking the two diagonals. This shows that any cycle can be made into one of length 6 for this case.

So for the case of 2 interior points, we already have $O(n^4)$ by enumerating 4 points on the hull and checking if i_1, a, z, i_2, z', a' is an alternating cycle. To optimize it, we can do a two-pointers algorithm, that finds the line segments az and $z'a'$ which intersect i_1i_2 , and which maximize / minimize the angle between segments i_1i_2 and az . These line two segments are always better than any other two line segments, because together they simultaneously minimize both the angles at $\angle a'i_1a$ and $\angle zi_2z'$. If these two line segments do not work, there is no alternating cycle.

This can be implemented in $O(n \log(n))$: First, construct the convex hull, then find the interior points. Based on the number of them, do casework, and for 2 interior points do a two-pointers algorithm in $O(n)$ time.

Proof for 3 interior points

First, let's draw a triangle composed of the 3 interior points. The lines extending the edges of the triangle subdivide the plane into 6 sections in which convex hull points could lie. Given any convex hull that completely encloses the triangle, we can actually take ≤ 6 points from it and remove the rest:

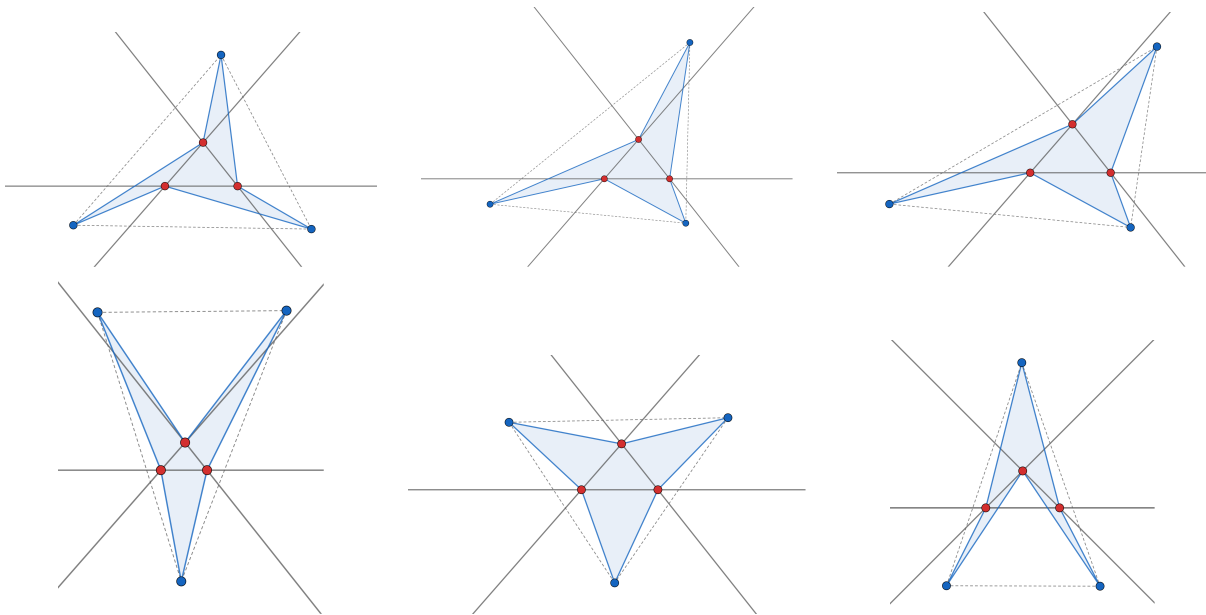


Reducing to a case with ≤ 6 points on the hull.

For pairs of neighbouring plane divisions $(0, 1)$, $(2, 3)$, $(4, 5)$ we can imagine gluing them together, leaving us with a division into 3 regions. We take the first and last point of the hull that reside in such a region of the plane. There can be 0 or 1 points inside the region, in which case we take no points in this piece, or 1. The resulting convex polygon actually encloses the triangle. To see this, we can prove that for each edge of this polygon, the triangle does not intersect this edge and does not lie on the wrong side of this edge. For original edges of the convex hull, this is true by definition, which means that all edges going in between two different regions, they are all hull edges, so they satisfy the criterium. For an edge inside a plane division, it only shortcuts a piece of the space that before had no interior points in it, because it fully inside this region, so it also does not intersect the triangle. This proof can also be turned into a linear time algorithm to find an $O(1)$ sized subset to check with brute force.

So for proving that for all point sets with ≥ 3 interior points there exists a 6-alternating cycle, it suffices to check finitely many cases with from 3 to 6 convex hull points, and exactly 3 interior points.

For 3 points on the convex hull, we can enumerate all possible ways to distribute 3 points into the 6 regions, ignoring rotations, mirrorings and cases which do not enclose the triangle. This gives 6 distinct ways, and in each of them you can find a 3-sided star, as show in the following figure. All the angles are guaranteed to be as shown in the picture, because of in which region of the plane the hull points lie, or the fact that the hull must enclose the triangle.



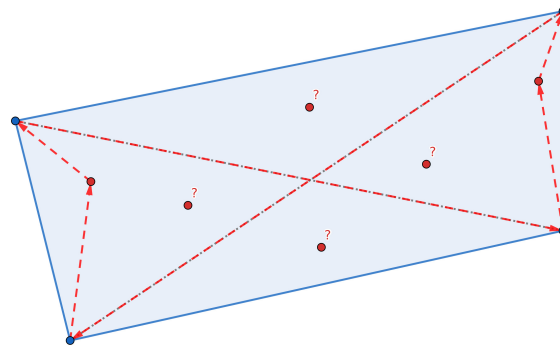
All

cases for 3 points on the hull, 3 points on the interior.

A trick to reduce the number of cases is to make the cases satisfy an extra constraint: If a convex hull has some point which can safely be removed, while keeping the interior triangle still interior, it could instead be reduced to a smaller case, so these cases can also be safely ignored.

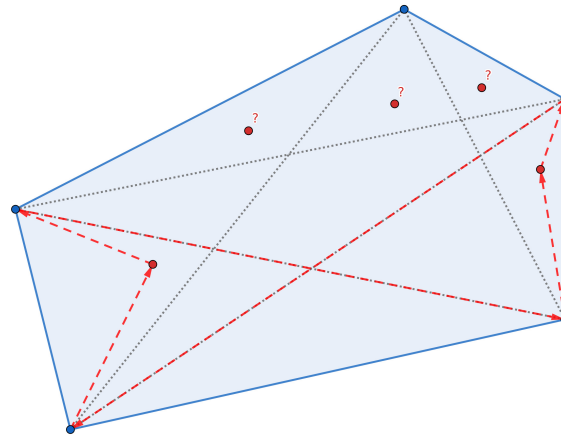
For 4 until 6 points on the hull, we can use our trick:

For 4, let's draw 4 points in convex position, and draw also draw all diagonals between them. The interior points can be divided inside 4 different triangles. There must be two interior points in opposite triangles, otherwise a point could be removed from the hull. These two points together with the triangles they are in form a butterfly.



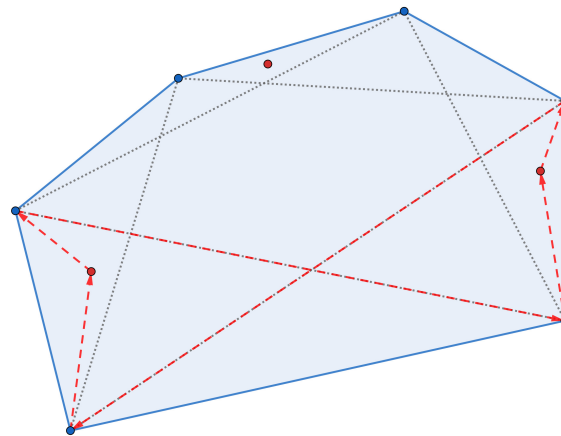
4 points on the hull case.

For 5 points on the hull, we can draw a similar picture, with all diagonals of the hull. There must be two of the interior points that lie inside two outer triangles, which are distance 2 apart. This is because for each point on the hull, when we remove it, we would get a new convex polygon of size 4. To make sure this cannot be done, there must a point inside the triangle neighbouring the 4-sided polygon.



5 points on the hull case.

For 6 points on the hull, there are 6 points that are not allowed to be removed from the hull, and there is only one way (up to rotation) to place points inside the regions to make sure this does not happen. The reasoning for this is also that certain triangles must have a point in them, and all these constraints together force the 3 points to the outside at regular intervals. And also in this way, you can immediately spot a butterfly.



6 points on the hull case. This is the only possible configuration of the points, and it also gives a butterfly.

Problem H. Game Design

We are given a subset of special elements S and need to count the number of permutations p such that for each i , at least one of the following holds: (a) $p_i < i$ or (b) $p_i \neq i$ and p_i is special.

Suppose we drop the restriction that $p_i \neq i$ in (b). Then we have a simple procedure that can generate any valid permutation. We build the permutation from left to right and maintain a pool of possible elements that p_i can be legally set to. Initially, the pool consists of the special elements. For each i :

- Pick and remove any element from the pool, set p_i to that element.
- If i is not special, add i to the pool.

The number of ways to pick p_i is simply the size of the pool. The pool stays the same size after setting p_i if i is non-special, and the size is decreased by one if i is special. This allows us to count the number of such permutations. If S consists of elements $i_1 < i_2 < \dots < i_k$, then the number of such permutations is

$$k^{i_1}(k-1)^{i_2-i_1}(k-2)^{i_3-i_2} \dots 1^{i_k-i_{k-1}} 0^{n-i_k}.$$

Now, let's move back to the original problem. We'll try to use the inclusion-exclusion principle to modify this solution for the original problem. For any subset $T \subseteq S$, let m_T be the number of permutations p such that:

- if $i \notin T$, then $p_i < i$ or p_i is special;
- if $i \in T$, then $p_i = i$.

That is, m_T is the number of permutations such that the $p_i \neq i$ rule is explicitly violated for a given set T of special indices (but it may also be violated for other special i). The inclusion-exclusion principle states that then, the answer to the problem is

$$\sum_{T \subseteq S} (-1)^{|T|} m_T. \quad (1)$$

Indeed, let's take the answer given by the initial algorithm. This includes permutations where $p_{i_1} = i_1$, so subtract the number of such permutations. Similarly, subtract the number of permutations where $p_{i_2} = i_2$. Now we've doubly subtracted the permutations where both $p_{i_1} = i_1$ and $p_{i_2} = i_2$ hold. Add the number of them back and so on. Eventually you'll arrive at the formula (1).

Computing $(-1)^{|T|} m_T$ is similar to the algorithm above. Start with a pool of size $|S \setminus T|$ and set the initial answer to be $(-1)^{|T|}$. Then

- If $i \in T$, do nothing.
- If $i \in S \setminus T$, multiply the answer by the pool size and reduce the pool size by 1.
- Otherwise, multiply the answer by the pool size.

Calculating the answer for each $T \subseteq S$ is obviously too slow, but we can use dynamic programming to calculate it all "in parallel". Set $\text{dp}[0][j] = 1$ if $j \equiv |S| \pmod{2}$ and -1 otherwise. Set $\text{dp}[i][j] = 0$ for $i > 0$. Now for each i from $1, \dots, n$ and each j from $0, \dots, |S|$:

- If $i \in S$, add $j \cdot \text{dp}[i-1][j]$ to $\text{dp}[i][j-1]$ and $\text{dp}[i-1][j]$ to $\text{dp}[i][j]$.
- Otherwise, add $j \cdot \text{dp}[i-1][j]$ to $\text{dp}[i][j]$.

The answer to the problem will be in $\text{dp}[n][0]$.

Problem I. Geometry Hacking

For beginners in computational geometry, the mistake Jeroen makes is quite a common one. But why does the algorithm presented fail? The problem is when a point of the polygon lies on the horizontal ray, but both the edges connected to this point come from below/above the ray. This adds 3 to ans, but that is wrong, as the ray is only glancing the polygon here, not actually passing from inside to outside, or from the outside to the inside. Another way to get the same wrong behavior is by adding a horizontal edge that lies exactly on the horizontal ray, and with the right way to connect the other edges to this horizontal edge, it also breaks the algorithm.

All these edgecases do not occur when the polygon is convex, as then the ray will only hit the polygon exactly once, when the query point is inside, and during this collision it will add $+1$ or $+3$ to ans depending on whether a point of the polygon is hit or not.

For a polygon with n corners, we can estimate its area with Pick's theorem: $A = i + \frac{b}{2} - 1$, where i is the number of interior points, and b is the number of points on the boundary. We know $b \geq n$, so we get $A \geq \frac{n}{2} - 1 + i$. Point $(0,0)$ is strictly inside the polygon, so $i \geq 1$. Polygons with $n = 3$ are triangles, which are convex, so would never make the algorithm fail.

So $n \geq 4$, and the area of the polygon must be $A \geq 1 + \frac{4}{2} - 1 = 2$. This lowerbound is achievable with an infinite family of polygons with 4 corners, so actually the k smallest area polygons can always be chosen

to be just k polygons out of such an infinite family. The easiest way to construct a family yourself is by checking when the lowerbound is tight. The lowerbound of 2 is only tight when there are no interior points except $(0, 0)$, there are no other points on the boundary except the corners of the polygon, and the polygon has exactly 4 corners. Also, there must be a point of the polygon lying on the ray, and the two edges must go down from there. By playing around with all these conditions on a piece of paper, you can, for example, find the following family of quadrilaterals: $(2, 0), (1, 2i + 1), (-2, 4i + 1), (1, -2i)$, with $i = 1, 2, 3, \dots$

Problem J. Non-Interactive Nim

Recall that a game of Nim is in a losing state if and only if $a_1 \oplus a_2 \oplus \dots \oplus a_n = 0$, where $\oplus$ denotes the bitwise XOR. Indeed:

- At the end of the game, all piles are empty and the total XOR is 0.
- If the total XOR is 0, any move you make will change the XOR to nonzero, because you have to change exactly one a_i .
- If the total XOR is nonzero, it is always possible to make a move that makes the XOR zero again: let k be the highest set bit of the total XOR. Find an a_i such that the k -th bit is set in a_i . Unset the k -th bit of a_i , then change the lower bits of a_i so that the XOR becomes 0. Notice that there is exactly one way to correctly set the lower bits of a_i . This definitely reduces a_i , so this is a valid Nim move.

Now we can see that the opponent has only one optimal move if and only if after your move, there is only one a_i with the k -th bit set, where k is the highest bit in the total XOR. The highest bit in the total XOR must then also have been the highest bit you changed in the your operation.

The solution to the problem is actually very simple:

- Let k be the highest set bit among all a_i .
- If there are more than two elements with the k -th bit set, report that the problem is unsolvable.
- Let a_i and a_j be the only elements with the k -th bit set. Set $a_i \leftarrow 0$, i.e. remove all stones from the i -th pile. Your opponent will XOR a_j with the old value of a_i .

Repeat until the array becomes zero.

Now we prove that this solution will work on all instances where the problem is solvable. If we incorporate the opponent's moves into your moves, this allows us to reformulate the problem. Given is an array $a_1, \dots, a_n$. Consider the following **operation**:

- Pick a bit k such that there are exactly two elements a_i and a_j with the k -th bit set.
- Set $a_i \leftarrow a_i \oplus x$ and $a_j \leftarrow a_j \oplus x$ for some x with highest set bit x . In other words, unset the k -th bit in a_i and a_j and change the bits $0, 1, \dots, k - 1$ in a_i and a_j in any way, subject to parity.

Can you make the entire array zero using this operation?

Consider the following process. Build a graph on n vertices and initially no edges. For each k from 60 to 0, count the number of connected components with an odd number of vertices i where $a_{ik} = 1$.

- If there are 0 such connected components, do nothing.
- If there are 2 such connected components, add an edge between them.
- If there are 4 or more such connected components, the process fails.

We will prove that the problem is solvable if and only if this process doesn't fail.

Lemma 1. Suppose there is a sequence of operations $(k_1, i_1, j_1, x_1), (k_2, i_2, j_2, x_2), \dots$ that makes the array into 0, such that $k_1 > k_2 > \dots$. Then this process doesn't fail.

Proof. No operation can change the total parity of a connected component.

Lemma 2. Suppose this process doesn't fail. Then, there is a sequence of operations $(k_1, i_1, j_1, x_1), (k_2, i_2, j_2, x_2), \dots$ that makes the array into 0. This sequence satisfies $k_1 > k_2 > \dots$.

Proof. Make a DSU with n elements. We'll run the process, maintaining the following invariants at all times:

- If vertex i is the representative of its connected component C , then $a_{ik} = 1$ for exactly those bits k for which there is an odd number of k -th bits in C .
- If vertex i is not the representative of its connected component, then $a_i = 0$.
- After step k , $a_{ik} = 0$ for all i .

Run the process. Suppose that for some k there are 2 connected components with an odd number of set k -th bits. Let their representatives be i and j . Append the operation (k, i, j, a_j) and make i the representative of the now united component. It is easy to see that doing this maintains the invariants and at the end of the process, the array is 0.

Lemma 3. Suppose there is a sequence of operations $(k_1, i_1, j_1, x_1), (k_2, i_2, j_2, x_2), \dots$ that makes the array into 0. Then this process doesn't fail.

Proof. If the sequence k_i is decreasing, then we already know that the process doesn't fail by lemma 1. Let p be the last position where $k_p < k_{p+1}$. Let a' be the array before the operation (k_p, i_p, j_p, x_p) and a'' be the array after it.

We know by lemma 1 that the process doesn't fail on a'' . We will show that the process doesn't fail on a' either. Let's compare the process on a' with the process on a'' . Steps before the k_p -th are identical. Because (k_p, i_p, j_p, x_p) is a valid operation, we know that a'_{i_p} and a'_{j_p} are the only elements of a' that have the k_p -th bit set. Therefore, no elements in a'' have that bit set. The process of a' does nothing on step k . If i_p and j_p are in different connected components in the process of a' , then they will be the only two connected components with an odd number of k_p -th bits and will be connected by an edge in the k_p -th step. If i_p and j_p are in the same connected component in the process of a' , then the k_p -th step also does nothing.

Thus, after the k_p -th step, the connected components in the processes of a' and a'' are identical, except for the fact that the connected components containing i_p and j_p are joined. If the operation (k_p, i_p, j_p, x_p) doesn't make any changes in the lower bits of a'_{i_p} and a'_{j_p} , then the rest of the process will also be identical, except for possibly skipping a step where i_p and j_p are joined in the process of a'' . If the operation does make changes in the lower bits, this also doesn't affect anything, as i_p and j_p are in the same connected component and XOR-ing both elements with some x doesn't change the parity of the number of set k -th bits.

Therefore, the process doesn't fail on a' and by lemma 2, there exists a sequence of operations with descending k -s that makes a' into 0. Thus, we can replace the suffix $(k_p, i_p, j_p, x_p), (k_{p+1}, i_{p+1}, j_{p+1}, x_{p+1}), \dots$ with such a decreasing sequence. This reduces the number of adjacent operations with $k_q < k_{q+1}$. By repeating this transformation, we will eventually have a sequence of operations with a descending sequence of k -s, which implies that the process doesn't fail.

Now we have shown that if there is a sequence of operations that makes the array into 0, we can use the algorithm of lemma 2 to turn the array into 0, which is precisely the solution presented in the beginning.

Problem K. String and Nails

This problem is asking to repeatedly build the convex hull of a set of points and erasing one of the corners of the convex hull (points lying on an edge of the convex hull are not allowed). Let's look at the point

which minimizes (x, y) lexicographically. There will not be any points with smaller x , and all points with the same x will have bigger y . This means that the extreme point in the direction $(-1, -\varepsilon)$, for ε tiny, will always be this point, and it also will be the unique extreme point. This means this point must be a corner of the convex hull.

So the solution becomes: sort the points in increasing order of the pairs (x, y) lexicographically. Then output the first $n - 1$ points.

Problem L. All-You-Can-Eat

This is an example of an online algorithm problem (in its non-competitive programming definition): you're looking at a version of a classical problems where the input is revealed one-by-one, and you have to make decisions after each piece of input is revealed. You seek to maximize the worst-case ratio between the result you achieved and the result you could have achieved if you had known everything ahead of time.

This problem is likely well-known to people who are studying or researching such things, but since there is almost no representation of problems like this in competitive programming, I thought it might make an interesting introduction.

Algorithms in this world tend to be very simple, but can be surprisingly hard to come up with. Here is one possible optimal solution. Let's call item i *small* if $a_i < 400$, *large* if $400 \leq a_i < 600$ and *massive* if $600 \leq a_i$.

For each i , then:

1. If the sum of items we have is at least 600, ignore.
2. Otherwise, if i is massive, take it and discard everything else.
3. Otherwise, if i can be added without discarding anything, take it.

If we have reached this point, we know that i is large — we had a sum of at most 599 before, and we could not add i without discarding anything.

4. Otherwise, if there are no large items in the table, take i and discard other items arbitrarily until it can be added without exceeding capacity.

If we have reached this point, we know that i is large and there is exactly one other large item in the bin (two would mean that their sum is at least 800, and we would have ignored that in the first step already). Let the other large item have size ℓ .

5. Otherwise, if $\ell + a_i \leq 1000$, take i and discard everything except for the other large item in the bin.
6. Otherwise, if $a_i < \ell$, take i and discard the other large item.
7. Otherwise, ignore.

If the sum of items is at least 600, we immediately succeed — the optimal solution has sum at most 1000. Notice that cases 2 and 5 take us to that state immediately. In fact, notice that so does case 4: in that case, before the last item is discarded, the sum of items exceeds 1000; the last item reduces this by less than 400, therefore the sum exceeds 600.

It only remains then to analyze sequences where our algorithm only visits cases 3, 6 and 7. In that case, the input consists of some small items with sizes $s_1, s_2, \dots, s_p$ and some big items with sizes $\ell_1, \ell_2, \dots, \ell_q$, where $\ell_i + \ell_j > 1000$ for all $i \neq j$. Observe that we never discard any small items in this case. Our algorithm gets score

$$s_1 + s_2 + \dots + s_p + \min_i \ell_i,$$

while the optimal solution is to take one of the large items ℓ_j and some subset of the small items. Let the total sum of the small items be S . Then, the ratio is

$$\frac{s_1 + s_2 + \cdots + s_p + \min_i \ell_i}{S + \ell_j} \geq \frac{s_1 + s_2 + \cdots + s_p + \min_i \ell_i}{s_1 + s_2 + \cdots + s_p + \ell_j} \geq \frac{\min_i \ell_i}{\ell_j} \geq \frac{400}{600} > 0.6.$$

By adjusting the boundaries, it is possible to get a ratio of at least $\frac{2}{1+\sqrt{5}}$ in every case, and it is possible to show that no algorithm can achieve a higher ratio in every case.